



UNIVERSITÉ
LIBRE
DE BRUXELLES

08 Mai 2026

Document Protocole

Étudiants:

ADIYA, C.

BENZIANI, N.

BEATRIZ DE ASSUNÇÃO, T.

CONTULIANO BRAVO, S.

KANOUN, A.

Assistant académique:

HUGO CALLEBAUT

Table des matières

1 Introduction technique	1
1.1 Structure des paquets	1
2 Tableau des messages	2
2.1 Authentification	2
2.2 Lobby	2
2.3 Projet	5
2.4 Carte	6
2.5 Chat	10

1 Introduction technique

Le protocole de communication est implémenté selon une architecture client-serveur via des communications par sockets TCP. Ici, TCP est privilégié pour garantir que les données soient bien transmises dans l'ordre et qu'aucun paquet ne soit perdu, ce qui est crucial pour la synchronisation en temps réel.

1.1 Structure des paquets

Chaque paquets envoyés suivra la structure suivantes:

- En-tête (header):
 - ID du message
 - taille des données
- Données (payload):
 - ID du layer
 - ...

2 Tableau des messages

2.1 Authentification

Message	Sens	Arguments	Description
<code>AUTH_LOGIN_REQ</code>	client → serveur	<code>string pseudo</code> <code>string password</code>	Demande de se connecter
<code>AUTH_REGISTER_REQ</code>	client → serveur	<code>string pseudo</code> <code>string password</code>	Demander de s'enregistrer
<code>AUTH_RESULT</code>	serveur → client	<code>uint8 succes</code>	Autorise/refuse la connexion/enregistrement

2.2 Lobby

Message	Sens	Arguments	Description
<code>LOB_INFO_USER_REQ</code>	client → serveur	<code>string pseudo</code> <code>string password</code>	Demande les informations du profil
<code>LOB_INFO_USER_REP</code>	serveur → client	<code>string pseudo</code> <code>string password</code>	Envoi les informations du profil
<code>LOB_CREATE_PROJECT_REQ</code>	client → serveur	<code>uint8 role</code> , <code>uint scale</code> <code>uint height</code> , <code>uint width</code>	Créer un projet
<code>LOB_GET_PROJECT_DATA_REQ</code>	client → serveur	<code>uint PROJECT_ID</code>	Demande les données d'un projet
<code>LOB_DEL_PROJECT_REQ</code>	client → serveur	<code>uint PROJECT_ID</code>	Supprime un projet de la base de donnée
<code>LOB_PROJECT_DATA_REP</code>	serveur → client	<code>uint PROJECT_ID</code> <code>uint32_t data_size</code> <code>QByteArray data</code>	Envoi les données d'un projet
<code>LOB_SHARE_PROJECT_REQ</code>	client → serveur	<code>uint PROJECT_ID</code> <code>uint8_t shared_role</code>	Demande un code de partage
<code>LOB_SHARE_PROJECT_REP</code>	serveur → client	<code>uint code</code>	Envoi un code de partage
<code>LOB_JOIN_PROJECT_REQ</code>	client → serveur	<code>uint code</code>	Demande de rejoindre un projet
<code>LOB_JOIN_PROJECT_REP</code>	serveur → client	<code>bool success</code>	Confirme de la validité du token de partage
<code>LOB_EXPORT_PNG_PROJECT_REQ</code>	client → serveur	<code>uint project_id</code>	Demande la transformation de la zone de base du projet dans le format souhaité
<code>LOB_EXPORT_INT_OBJ_LAYER_REQ</code>	client → serveur	<code>uint layer_id</code> <code>uint project_id</code> <code>string sprite_path</code> <code>uint32 sprite_size</code> <code>uint8[] raw_png_data</code>	Demande tout les objets se trouvant sur la couche en fonction de l'indice du projet
<code>LOB_EXPORT_INT_OBJ_LAYER_REP</code>	serveur → client	<code>uint layer_id</code> <code>uint project_id</code> <code>uint32 sprite_size</code> <code>uint8[] raw_png_data</code>	Demande tout les objets se trouvant sur la couche en fonction de l'indice du projet
<code>LOB_EXPORT_INT_OBJ_LAYER_REQ</code>	client → serveur	<code>uint layer_idx</code> <code>uint project_idx</code> <code>string sprite_path</code> <code>uint32 sprite_size</code> <code>uint8[] raw_png_data</code>	Demande tout les objets se trouvant sur la couche en fonction de l'indice du projet
<code>LOB_EXPORT_INT_OBJ_LAYER_REP</code>	serveur → client	<code>uint layer_id</code>	Envoie tout les objets se trouvant sur la couche

Message	Sens	Arguments	Description
LOB_EXPORT_INT_PIX_LAYER_COORD_REQ	client → serveur	int x int y	Demande coordonnées de la couche
LOB_EXPORT_INT_PIX_LAYER_COORD_REP	serveur → client	int x int y	Envoie coordonnées de la couche
LOB_IMPORT_PROJECT_REQ	client → serveur	QByteArray project_data	Créer un projet vide avec la taille et l'échelle. Ensuite update les couches en fonction de la lecture du fichier JSON lu depuis un fichier de sauvegarde (LOB_UPLOAD_OBJ_LAYER, LOB_UPLOAD_PIX_LAYER)
LOB_IMPORT_PROJECT_REP	serveur → client	bool succes int new_project_id	Renvoie la confirmation de la création d'un projet avec son id
LOB_EXPORT_NATIVE_PROJECT_REQ	client → serveur	uint projectId std::string destinationPath	Demande au serveur de créer un fichier zip d'un projet voulu
LOB_EXPORT_NATIVE_PROJECT_REP	serveur → client	std::string projectName std::string destPath QByteArray zipArchive	Le serveur renvoie une réponse qui contient le fichier zip, son nom et l'endroit pour le sauvegarder, pour que le handler puisse le faire
LOB_OBJECT_LAYER_REQ	client → serveur	int num_objects std::vector <sf::RenderTexture> Object	Demande couche objet
LOB_OBJECT_LAYER_REP	serveur → client	int num_objects std::vector <sf::RenderTexture> Object	Envoie couche objet
LOB_PIXEL_LAYER_REQ	client → serveur	sf::Image Layer	Demande couche Pixel
LOB_PIXEL_LAYER_REP	serveur → client	sf::Image Layer	Envoie couche Pixel
LOB_UPLOAD_OBJ_LAYER_REQ	client → serveur	int project_id int layer_index std::vector <Object> list_objects	Le client envoie la liste de tous les objets en fonction des couches qu'il a trouvé dans le JSON
LOB_UPLOAD_OBJ_LAYER_REP	serveur → client	bool success	Confirme la mise à jour des couches du projet
LOB_UPLOAD_PIX_LAYER_REQ	client → serveur	int layer_index int project_id uint8[] raw_png_data	Le client envoie au serveur la couche objet (png) sous forme de tableau d'octet pour le stocker.
LOB_UPLOAD_PIX_LAYER_REP	serveur → client	bool success	confirmation que la sauvegarde à été faite
LOB_PROJECT_LIST_REQ_REQ	client → serveur	int num_proj std::vector <string> name	demande la liste des projets
LOB_PROJECT_LIST_REQ_REP	serveur → client	int num_proj std::vector <string> name	envoi la liste des projets
LOB_QUIT_PROJECT_REQ	client → serveur	int project_id	informe le server que l'on retourne au lobby pour ne plus recevoir les données du projets
LOB_RENAME_PROJECT_REQ	client → serveur	int project_id std::string newName	envoie le nouveau nom du projet dont l'id = project_id
LOB_RENAME_PROJECT_REP	serveur → client	long long user_Id int project_id std::string newName bool success	renvoie au user le nouveau nom, l'id du nouveau projet et la confirmation du renommage
LOB_DUPLICATE_PROJECT_REQ	client → serveur	uint project_id std::string newName	Envoie le projet à dupliquer et le nouveau nom du duplicata
LOB_DUPLICATE_PROJECT_REP	serveur → client	uint user_Id uint project_id	Confirme la création du nouveau projet project_Id de nom new-

Message	Sens	Arguments	Description
		std::string newName bool success	Name dont le user_Id est propriétaire

2.3 Projet

Message	Sens	Arguments	Description
PROJ_GET_MEMBERS_REQ	client → serveur	uint project_id	Demander les membres d'un projet
PROJ_GET_MEMBERS_REP	serveur → client	vector<MemberEntry>	Recevoir tous les membres participants au projet
PROJ_CHANGE_ROLE_REQ	client → serveur	uint targetId uint projectId int8_t role	Requête pour changer le role de l'utilisateur 'targetId' dans le projet 'projectId'
PROJ_CHANGE_ROLE_REP	serveur → client	bool success uint targetId uint projectId int8_t role	Recevoir le nouveau rôle si le changement à été effectué dans le serveur
PROJ_LEAVE_PROJ_REQ	client → serveur	uint projectId	Requête pour pouvoir quitter le projet afin de mettre à jour le serveur
PROJ_KICK_USER_REQ	client → serveur	uint projectId uint targetId	Requête pour supprimer l'accès au projet d'un participant 'targetId'
PROJ_KICK_USER_REP	serveur → client	bool success uint targetId uint projectId	Réponse du serveur pour informer les autres participants que celui qui à été éjecter n'est plus là, et pour informer la target de son expulsion

2.4 Carte

Message	Sens	Arguments	Description
MAP_CREATE_LAYER_REQ	client → serveur	uint project_id LayerType layer_type	Créer une couche
MAP_CREATE_LAYER_REP	serveur → client	uint project_id LayerType layer_type	Ajoute à tout les utilisateurs la nouvelle couche
MAP_RENAME_LAYER_REQ	client → serveur	uint project_id uint layer_id str name	Renomme une couche
MAP_RENAME_LAYER_REP	serveur → client	uint project_id uint layer_id str name	Informe tout les utilisateurs d'un nouveau nom pour une couche
MAP_REMOVE_LAYER_REQ	client → serveur	uint project_id uint layer_id	Retire une couche
MAP_REMOVE_LAYER_REP	serveur → client	uint project_id uint layer_id	Informe tout les utilisateurs de la suppression d'une couche
MAP_MOV_LAYER_REQ	client → serveur	uint project_id uint layer_id int x int y	Déplace une couche
MAP_MOV_LAYER_REP	serveur → client	uint project_id uint layer_id int x int y	Actualise la position d'une couche chez tout les utilisateurs
MAP_ORGANIZE_LAYER_UP_REQ	client → serveur	uint project_id uint layer_id	Déplace une couche dans la pile des couches
MAP_ORGANIZE_LAYER_UP_REP	serveur → client	uint project_id uint layer_id int new_depth	Informe tout les utilisateurs du déplacement d'une couche dans la pile des couches
MAP_ORGANIZE_LAYER_DOWN_REQ	client → serveur	uint project_id uint layer_id	Déplace une couche dans la pile des couches
MAP_ORGANIZE_LAYER_DOWN_REP	serveur → client	uint project_id uint layer_id	Informe tout les utilisateurs du déplacement d'une couche dans la pile des couches
MAP_IMPORT_SPRITE_REQ	client → serveur	uint project_id str file_name uint32 file_size uint8[] file	Importe un nouveau sprite utilisable (si le fichier est trop gros il faut le fragmenter)
MAP_IMPORT_SPRITE_REP	serveur → client	uint project_id str path uint32 file_size uint8[] file	Ajoute à tout les utilisateurs le nouveau sprite importé
MAP_PUT_SPRITE_REQ	client → serveur	uint project_id uint sprite_id uint layer_id uint x uint y	Place un sprite
MAP_PUT_SPRITE_REP	serveur → client	uint project_id str sprite_name uint layer_id uint x uint y	Place le sprite chez tout les utilisateurs
MAP_ERASE_SPRITE_CIRCLE_REQ	client → serveur	uint project_id uint layer_id uint x uint y float taille	Utilise la gomme pour sprite de forme circulaire

Message	Sens	Arguments	Description
MAP_ERASE_SPRITE_CIRCLE_REQ	serveur → client	uint project_id uint layer_id uint x uint y float taille	Retire le sprite chez tout les utilisateurs
MAP_ERASE_SPRITE_SQUARE_REQ	client → serveur	uint project_id uint layer_id uint x uint y float taille	Utilise la gomme pour sprite de forme carré
MAP_ERASE_SPRITE_SQUARE_REQ	serveur → client	uint project_id uint layer_id uint x uint y float taille	Retire le sprite chez tout les utilisateurs
MAP_ERASE_SPRITE_DIAM_REQ	client → serveur	uint project_id uint layer_id uint x uint y float hauteur float largeur	Utilise la gomme pour sprite de forme losange
MAP_ERASE_SPRITE_DIAM_REQ	serveur → client	uint project_id uint layer_id uint x uint y float hauteur float largeur	Retire le sprite chez tout les utilisateurs
MAP_MOV_SPRITE_REQ	client → serveur	uint project_id uint layer_id vector<uint> sprite_ids int x int y	Déplace un sprite
MAP_MOV_SPRITE_REQ	serveur → client	uint project_id uint layer_id vector<uint> sprite_ids int x int y	Actualise la position du sprite chez tout les utilisateurs
MAP_ROTATE_SPRITE_REQ	client → serveur	uint project_id uint layer_id vector<uint> sprite_ids float angle vector<float> x vector<float> y	Tourne le sprite
MAP_ROTATE_SPRITE_REQ	serveur → client	uint project_id uint layer_id vector<uint> sprite_ids float angle vector<float> x vector<float> y	Actualise la rotation du sprite chez tout les utilisateurs
MAP_RESIZE_SPRITE_REQ	client → serveur	uint project_id uint layer_id vector<uint> sprite_ids vector<float> x vector<float> y float scale	Redimensionne le sprite
MAP_RESIZE_SPRITE_REQ	serveur → client	uint project_id uint layer_id vector<uint> sprite_ids vector<float> x	Redimensionne le sprite chez tout les utilisateurs

Message	Sens	Arguments	Description
		vector<float> y float scale	
MAP_PUT_PIXEL_CIRCLE_REQ	client → serveur	uint project_id uint layer_id uint x uint y float taille Uint8 red Uint8 green Uint8 blue Uint8 opacity	Utilisation du pixelBrush en forme de cercle
MAP_PUT_PIXEL_CIRCLE_REP	serveur → client	uint project_id uint layer_id uint x uint y float taille Uint8 red Uint8 green Uint8 blue Uint8 opacity	Utilisation du pixelBrush chez tout les autres utilisateurs
MAP_ERASE_PIXEL_CIRCLE_REQ	client → serveur	uint project_id uint layer_id uint x uint y float size	Utilisation de la gomme en forme de cercle
MAP_ERASE_PIXEL_CIRCLE_REP	serveur → client	uint project_id uint layer_id uint x uint y float size	Utilisation de la gomme en forme de cercle chez tout les autres utilisateurs
MAP_PUT_PIXEL_SQUARE_REQ	client → serveur	uint project_id uint layer_id uint x uint y float taille Uint8 red Uint8 green Uint8 blue Uint8 opacity	Utilisation du pixelBrush en forme de carré
MAP_PUT_PIXEL_SQUARE_REP	serveur → client	uint project_id uint layer_id uint x uint y float taille Uint8 red Uint8 green Uint8 blue Uint8 opacity	Utilisation du pixelBrush chez tout les autres utilisateurs
MAP_ERASE_PIXEL_SQUARE_REQ	client → serveur	uint project_id uint layer_id uint x uint y float size	Utilisation de la gomme en forme carré
MAP_ERASE_PIXEL_SQUARE_REP	serveur → client	uint project_id uint layer_id uint x uint y float size	Utilisation de la gomme chez tout les autres utilisateurs
MAP_PUT_PIXEL_DIAM_REQ	client → serveur	uint project_id uint layer_id uint x	Utilisation du pixelBrush en forme de losange

Message	Sens	Arguments	Description
		uint y float largeur float hauteur Uint8 red Uint8 green Uint8 blue Uint8 opacity	
MAP_PUT_PIXEL_DIAM_REQ	serveur → client	uint project_id uint layer_id uint x uint y float largeur float hauteur Uint8 red Uint8 green Uint8 blue Uint8 opacity	Utilisation du brush chez tout les autres utilisateurs
MAP_ERASE_PIXEL_DIAM_REQ	client → serveur	uint project_id uint layer_id uint x uint y float largeur float hauteur	Utilisation de la gomme en forme de losange
MAP_ERASE_PIXEL_DIAM_REQ	serveur → client	uint project_id uint layer_id uint x uint y float largeur float hauteur	Utilisation de la gomme chez tout les autres utilisateurs
ADD_SPRITE_REQ	client → serveur	uint userId uint projectId string pseudo sf::Texture sprite uint spriteId	Import d'un sprite
ADD_SPRITE_REQ	serveur → client	uint userId uint projectId string pseudo sf::Texture sprite uint spriteId	Import d'un sprite pour tout les autres utilisateurs
MAP_AUTOFILL_REQ	client → serveur	uint project_id uint layer_id uint count float rotation float size vector<string> asset_id vector<sf::Vector2f> positions	Utilisation de l'outil de Remplissage Automatique
MAP_AUTOFILL_REQ	serveur → client	uint project_id uint layer_id uint count float rotation float size vector<string> asset_id vector<sf::Vector2f> positions	Utilisation de l'outil de Remplissage Automatique chez tout les autres utilisateurs

2.5 Chat

Message	Sens	Arguments	Description
CHAT_MESSAGE_REQ	client → serveur	string auteur string message	Envoyer un message
CHAT_MESSAGE_REP	serveur → client	string auteur string message	Envoyer le message a tout les utilisateurs
CHAT_CONNECT_USER_REP	serveur → client	string user	Informe tout les utilisateurs de la connexion d'un participant
CHAT_DISCONNECT_USER_REP	serveur → client	string user	Informe tout les utilisateurs de la déconnexion d'un participant
CHAT_LOCK_LAYER_REP	serveur → client	string auteur int depth	Informe tout les utilisateurs du verrouillage d'une couche par un participant